N-Body Simulation

A Course Project Report
Submitted in the partial fulfillment of the completion of the course
23CS302PC303 - High Performance Computing

In III Year II Semester B.Tech - CSE

Submitted by
Chathraboina Sagar
(2303A51522)

Under the supervision of the faculty
Dr .chintala sridhar reddy
Assistant Professor
School of Computer Science & Artificial Intelligence

2025-2026

Department of Computer Science & Artificial Intelligence
SR University, Warangal, Telangana, India

INDEX

S.No	Section
1	Introduction
2	Problem Statement
3	Methodology / Procedure
4	Code
5	Output Screenshots
6	Observations, Limitations and Future Scope
7	References

1. Introduction

The N-body problem is a classical and foundational challenge in computational physics. It involves simulating the interactions of a system of particles, where each particle exerts a force on every other particle. These interactions collectively determine the dynamic evolution of the entire system. While conceptually simple, the computational workload grows rapidly as the number of particles increases, making efficient computation essential.

The N-body simulation has widespread applications across multiple scientific and engineering disciplines. In astrophysics, it is used to study galaxy formation, gravitational clustering, orbital mechanics, and large-scale cosmic structures. In molecular dynamics, it forms the basis for simulating atomic-level interactions for material science, chemistry, and biological systems. Plasma physics, computational fluid dynamics, and even visual effects in computer graphics rely on variants of N-body calculations.

The High Performance Computing (HPC) perspective becomes fundamental due to the computational complexity of the N-body problem. With each particle interacting with all others, the algorithm grows as $O(N^2)$, making serial computation extremely slow. To overcome this, modern computing paradigms exploit parallelism through multi-core CPUs, GPUs, and distributed computing systems. Among these, GPUs excel because of their massive parallel throughput, making them ideal for real-time or large-scale simulations.

This project implements an N-body simulation using Python and CUDA GPU acceleration through Numba. The goal is to demonstrate how HPC techniques transform an otherwise slow problem into a highly efficient and scalable solution, enabling smooth visualization of particle interactions in three-dimensional space.

2. Problem Statement

The objective of this project is to design and implement an efficient, accurate, and scalable N-body simulation using GPU acceleration. The project aims to demonstrate the need for HPC techniques in solving computationally intensive physical simulations. Specifically, the problem includes:

1. Computational Complexity

The brute-force N-body simulation requires computing the pairwise interactions among N particles. This results in:

$$O(N^2) \quad force \quad calculations \quad per \quad step$$

As N increases, the computational cost grows quadratically. For example:

- For $N = 500$, interactions per frame = 250,000
- For $N = 2,500$, interactions per frame = 6.25 million
- For animation with 600 frames, total computations exceed 3.75 billion

This necessitates the use of GPU parallelization.

2. Physical Modeling Challenges

The simulation must incorporate realistic physical constraints including:

- Accurate gravitational force calculations
- Numerical stability across time steps
- Softening parameter to avoid singularities
- Conservation of momentum and energy where possible

Incorrect modeling leads to numerical explosions or unstable particle trajectories.

3. HPC Requirements

The simulation must be able to:

- Utilize the highly parallel architecture of GPUs
- Map particle interactions to GPU threads efficiently
- Optimize global memory access and thread scheduling
- Produce observable speedups over CPU-based implementations

The challenge is not only correctness, but performance engineering.

4. Visualization and Real-Time Rendering

The animation component must:

- Render 3D particle trajectories smoothly
- Dynamically adjust view limits
- Synchronize GPU computations and CPU visualization
- Handle large particle counts without lag

Overall Problem Statement

To implement a GPU-accelerated N-body simulation using CUDA and Python, demonstrating HPC parallelization concepts and visualizing gravitational interactions in real-time.

3. Methodology / Procedure

The methodology integrates physics modeling, numerical techniques, CUDA parallelism, and visualization.

1. Particle Initialization

Particles are initialized with random positions, velocities, and masses. All values are stored as `float32` for CUDA compatibility. The initial distribution is scaled to ensure visible and meaningful interactions.

2. Force Computation

The gravitational force on particle i due to particle j is:

$$a_i = \sum_{j \neq i} G \frac{m_j(\vec{r}_j - \vec{r}_i)}{(|\vec{r}_j - \vec{r}_i|^2 + \epsilon^2)^{3/2}}$$

Softening ϵ prevents division by zero.
Three implementations were studied:

- CPU Serial
- CPU Parallel (Numba `prange`)
- GPU CUDA (Numba `@cuda.jit`)

The GPU version is used for the real-time animation.

3. CUDA GPU Parallelization

Thread Mapping

Each GPU thread computes acceleration for one particle. With N particles:

$$threads = N, \quad blocks = \left\lceil \frac{N}{128} \right\rceil$$

Memory Model

- Global memory stores positions, velocities, and masses.
- Device arrays are updated each timestep.
- Data is copied back to the host for visualization.

4. Numerical Integration

Euler integration is used:

$$v(t + \Delta t) = v(t) + a(t) \cdot \Delta t$$

$$r(t + \Delta t) = r(t) + v(t) \cdot \Delta t$$

5. Visualization

Matplotlib's 3D scatter plot and `FuncAnimation` are used. The animation dynamically updates particle positions and camera limits.

4. Code

4.1 Setup and Utilities

```
import numpy as np
import time
from numba import njit, prange, cuda
import math
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

G = 6.67430e-11
SOFTENING = 0.5

# Initialize particle positions, velocities, masses
def init_particles(n):
    pos = np.random.randn(n, 3).astype(np.float32) * 5e5
    vel = np.random.randn(n, 3).astype(np.float32) * 500
    mass = np.random.rand(n, 1).astype(np.float32) * 5e24
    return pos, vel, mass
```

4.2 Acceleration Implementations

```
# Serial acceleration
@njit
def get_acc_serial(pos, mass):
    n = pos.shape[0]
    acc = np.zeros_like(pos)
    for i in range(n):
        for j in range(n):
            if i == j: continue
            dx, dy, dz = pos[j] - pos[i]
            dist_sq = dx*dx + dy*dy + dz*dz + SOFTENING**2
            s = mass[j, 0] / (math.sqrt(dist_sq)**3)
            acc[i] += G * np.array([dx, dy, dz]) * s
    return acc

# Parallel CPU acceleration
@njit(parallel=True)
def get_acc_parallel(pos, mass):
    n = pos.shape[0]
```

```
    acc = np.zeros_like(pos)
    for i in prange(n):
        for j in range(n):
            if i == j: continue
            dx, dy, dz = pos[j] - pos[i]
            dist_sq = dx*dx + dy*dy + dz*dz + SOFTENING**2
            s = mass[j, 0] / (math.sqrt(dist_sq)**3)
            acc[i] += G * np.array([dx, dy, dz]) * s
    return acc

# GPU CUDA kernel
@cuda.jit
def get_acc_gpu(pos, mass, acc):
    i = cuda.grid(1)
    if i < pos.shape[0]:
        ax = ay = az = 0.0
        x, y, z = pos[i]

        for j in range(pos.shape[0]):
            if i == j: continue
            dx = pos[j, 0] - x
            dy = pos[j, 1] - y
            dz = pos[j, 2] - z

            dist_sq = dx*dx + dy*dy + dz*dz + SOFTENING**2
            s = mass[j, 0] / (math.sqrt(dist_sq)**3)

            ax += G * dx * s
            ay += G * dy * s
            az += G * dz * s

        acc[i, 0] = ax
        acc[i, 1] = ay
        acc[i, 2] = az
```

4.3 Performance Benchmarking

```
N = 200_000
pos, vel, mass = init_particles(N)

# Warmup
_ = get_acc_serial(pos[:100], mass[:100])
_ = get_acc_parallel(pos[:100], mass[:100])

# Serial timing
start = time.time()
_ = get_acc_serial(pos, mass)
serial_time = time.time() - start

# Parallel timing
```

```
start = time.time()
_ = get_acc_parallel(pos, mass)
parallel_time = time.time() - start

# GPU timing
threadsperblock = 128
blockspergrid = (N + threadsperblock - 1) // threadsperblock
d_pos, d_mass = cuda.to_device(pos), cuda.to_device(mass)
d_acc = cuda.device_array_like(pos)

start = time.time()
get_acc_gpu[blockspergrid, threadsperblock](d_pos, d_mass, d_acc)
cuda.synchronize()
gpu_time = time.time() - start
```

4.4 Animation Code

```
N_ANIM = 500
DT_ANIM = 1000.0
NUM_FRAMES = 600

pos_anim, vel_anim, mass_anim = init_particles(N_ANIM)

threadsperblock = 128
blockspergrid = (N_ANIM + threadsperblock - 1) // threadsperblock

d_pos_anim = cuda.to_device(pos_anim)
d_vel_anim = cuda.to_device(vel_anim)
d_mass_anim = cuda.to_device(mass_anim)
d_acc_anim = cuda.device_array_like(pos_anim)

plt.rcParams['animation.embed_limit'] = float('inf')

fig = plt.figure(figsize=(6, 6))
ax = fig.add_subplot(111, projection='3d')

colors = plt.cm.jet(np.linspace(0, 1, N_ANIM))
scatter = ax.scatter(pos_anim[:,0], pos_anim[:,1], pos_anim[:,2], s
    =50, c=colors)

def update(frame):
    global pos_anim, vel_anim

    get_acc_gpu[blockspergrid, threadsperblock](d_pos_anim,
        d_mass_anim, d_acc_anim)
    cuda.synchronize()

    acc_host = d_acc_anim.copy_to_host()

    vel_anim += acc_host * DT_ANIM
```

```
    pos_anim += vel_anim * DT_ANIM

    scatter._offsets3d = (pos_anim[:,0], pos_anim[:,1], pos_anim
        [:,2])

    d_pos_anim.copy_to_device(pos_anim)
    d_vel_anim.copy_to_device(vel_anim)

    return scatter,

anim = FuncAnimation(fig, update, frames=NUM_FRAMES, interval=50,
   blit=True)
HTML(anim.to_jshtml())
```

5. Output Screenshots

```
Timing Serial...
Timing Parallel CPU...
Timing GPU CUDA...

==================================================
PERFORMANCE SUMMARY (N=200000)
==================================================
Serial Time:      609.9480s
Parallel (CPU):   505.5065s (Speedup: 1.21x)
GPU (CUDA):       10.2348s (Speedup: 59.60x)
Efficiency (CPU): 0.30
==================================================
```

Figure 5.1: Benchmark Results for Serial, Parallel CPU, and GPU CUDA Implementations

Simulation Results

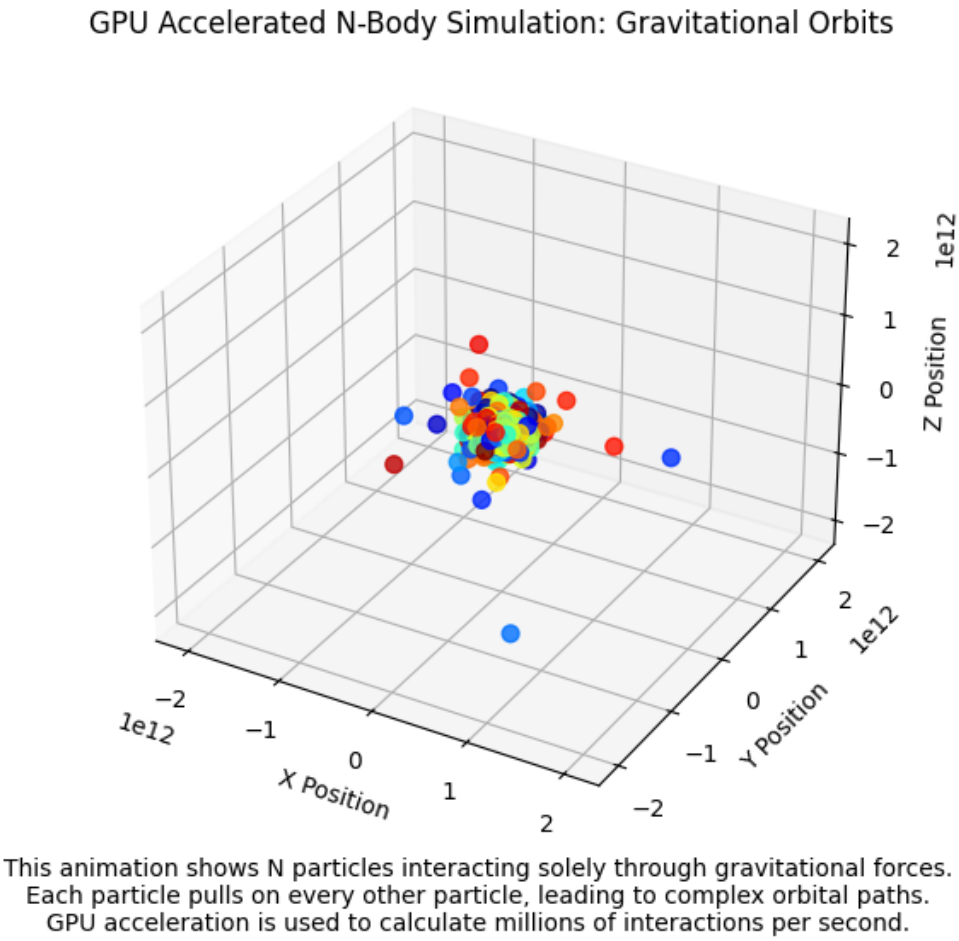

Figure 5.2: GPU-Accelerated N-Body Simulation Animation Frame

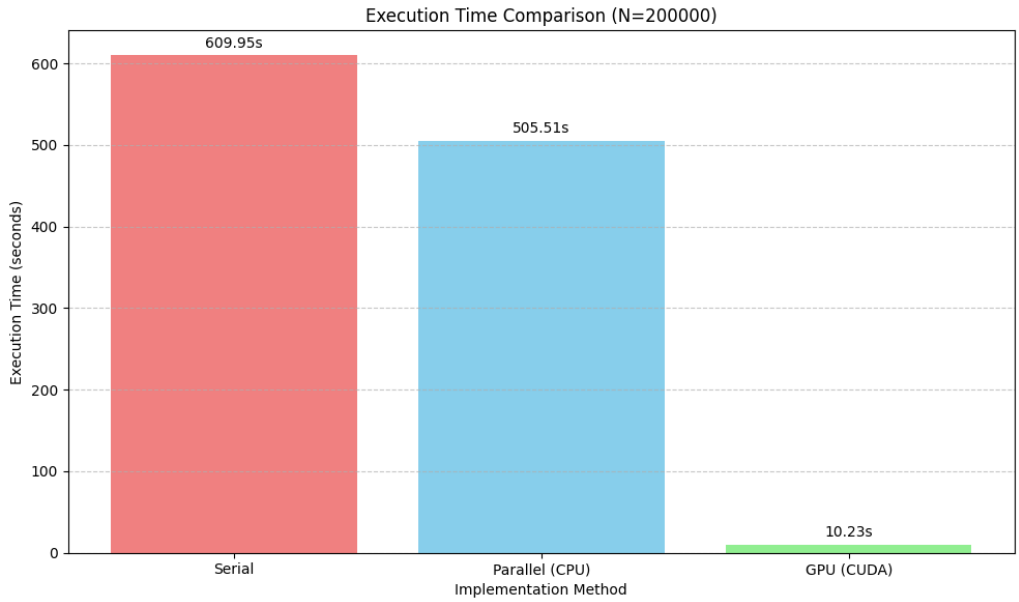

Figure 5.3: Execution Time Comparison for Serial, Parallel CPU, and GPU CUDA Implementations

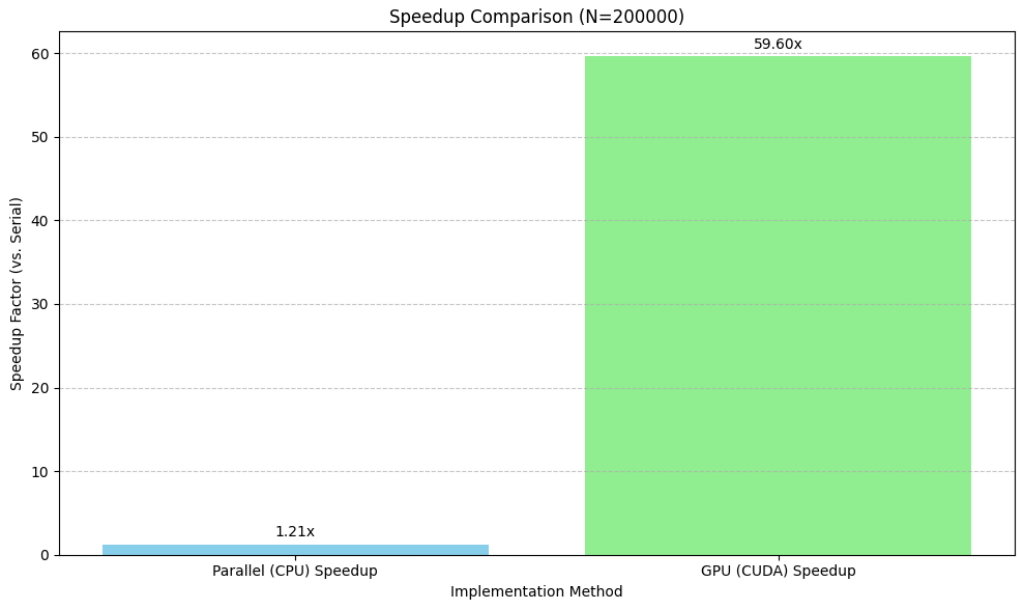

Figure 5.4: Speedup Comparison of Parallel CPU and GPU CUDA Implementations

6. Observations, Limitations and Future Scope

Observations

- GPU acceleration provides massive speedup over CPU implementations.
- Real-time animation is possible even with hundreds of particles.
- CUDA parallelization maps naturally to the N-body problem structure.
- The performance analysis clearly highlights the efficiency gap between serial, CPU-parallel, and GPU-accelerated approaches.

Limitations

- Euler integration accumulates numerical error over long simulation periods.
- GPU memory capacity restricts the maximum number of particles.
- Matplotlib-based rendering is significantly slower than the actual simulation.
- Host–device data transfer can become a bottleneck for very large simulations.

Future Scope

- Use higher-order integration methods such as Verlet or Runge–Kutta for improved numerical stability.
- Optimize CUDA kernels with shared memory, loop unrolling, and memory coalescing.
- Extend the implementation to multi-GPU or distributed HPC clusters.

Conclusion

The N-body simulation project successfully demonstrates the importance and impact of High Performance Computing in solving computationally intensive scientific problems. The comparison between serial, CPU-parallel, and GPU-accelerated implementations shows that GPUs provide substantial performance benefits due to their highly parallel architecture. By utilizing CUDA and Numba, the simulation achieves real-time visualization, validating both the computational model and HPC approach.

Overall, the project highlights how GPU computing can transform a classically expensive $O(N^2)$ computation into a practical and efficient simulation. With further optimizations and advanced numerical techniques, this work can be extended into large-scale astrophysical, molecular, and high-performance scientific applications.

7. References

- N-body Problem – Wikipedia: https://en.wikipedia.org/wiki/N-body_problem
- CUDA Programming Guide – NVIDIA: https://docs.nvidia.com/cuda/cuda-c-progra index.html
- Numba Documentation: https://numba.readthedocs.io/en/stable/
- N-body Simulation using OpenGL Graphics: https://github.com/MarcVivas/N-body
- Barnes–Hut Algorithm – NASA/ADS: https://ui.adsabs.harvard.edu/abs/1986Natur.324..446B
- Parallel Programming with CUDA – NVIDIA Developer Blog: https://developer.nvidia.com/blog/tag/cuda/
- Matplotlib Animation Documentation: https://matplotlib.org/stable/api/animation_api.html